

COMPLETE PROJECT GUIDE · IN SIMPLE WORDS

PR•VISION

A plain-language tour of the platform: how it detects misinformation early, verifies claims against real evidence, forecasts the spread with machine learning, and turns everything into a priority an analyst can act on — frontend, backend, pipeline, data and security, all without jargon.

AI-Powered Misinformation Early-Warning & Verification Platform

DETECT · VERIFY · ANALYZE · PREDICT · PRIORITIZE

SEPTEMBER 2026 · DOCUMENTATION EDITION

Contents

1. What is PR•VISION?	2
2. How the System Is Built (Big Picture)	3
3. The Toolbox: Technologies We Use	4
4. The Two Main Workspaces	6
5. The Verification Pipeline Step by Step	6
6. The Six Verdicts	9
7. Predicting Spread with XGBoost	9
8. Alerts, Priority Scores & Cases	10
9. Where the Data Lives	11
10. Security in Plain Words	12
11. Real Data vs Demo Mode: Honesty Rules	12
12. Run It Yourself + Project Map	13

1. What is PR•VISION?

PR•VISION is a web platform that works like a **smoke detector for fake news**. A smoke detector does not wait for the whole house to burn down; it watches the air all the time and screams the moment something smells wrong. In the same way, PR•VISION watches social media posts and suspicious content around the clock, checks them against real evidence, and warns analysts early, while a false story is still small enough to stop. The name says what it does: PR for perception and propagation, VISION for seeing clearly before others do.

Everything the platform does follows one straight line of five steps. Each step feeds the next, so a single suspicious post travels from “just noticed” to “here is what to do about it” without any human having to connect the dots by hand.

Step	What it means in plain words
Detect	Spot new posts and topics as they appear, and measure how fast they are spreading.
Verify	Break the content into checkable claims and compare each one against real evidence from trusted public sources.
Analyze	Score how risky the content is, how it is spreading, and who it is trying to reach.
Predict	Use machine learning to forecast how far and how fast it will spread in the next 30, 60 and 120 minutes.
Prioritize	Give every item a 0–100 priority score so analysts fix the biggest fire first, not the loudest one.

Table 1: The five-step mission of PR•VISION

6 verdict levels	4 alert levels	30/60/120 minute forecasts	0–100 priority score
----------------------------	--------------------------	--------------------------------------	--------------------------------

The platform is also deliberately **honest**. It never pretends an API is connected when it is not, never invents evidence, and never forces a clean “TRUE” or “FAKE” label onto something the evidence cannot settle. Those honesty rules are explained in Chapter 11, and they are the reason an analyst can actually trust what the screen says.

2. How the System Is Built (Big Picture)

The easiest way to understand the architecture is to picture a **restaurant**. The dining room is what you see: clean tables, a menu, everything presentable. That is the **frontend** running in your browser. The waiter walks between tables and the kitchen, taking orders and bringing food. That is the **backend server**. And in the kitchen there is a pantry full of ingredients plus the chefs who do the real work. That is the **data layer**: the database and the machine-learning models.

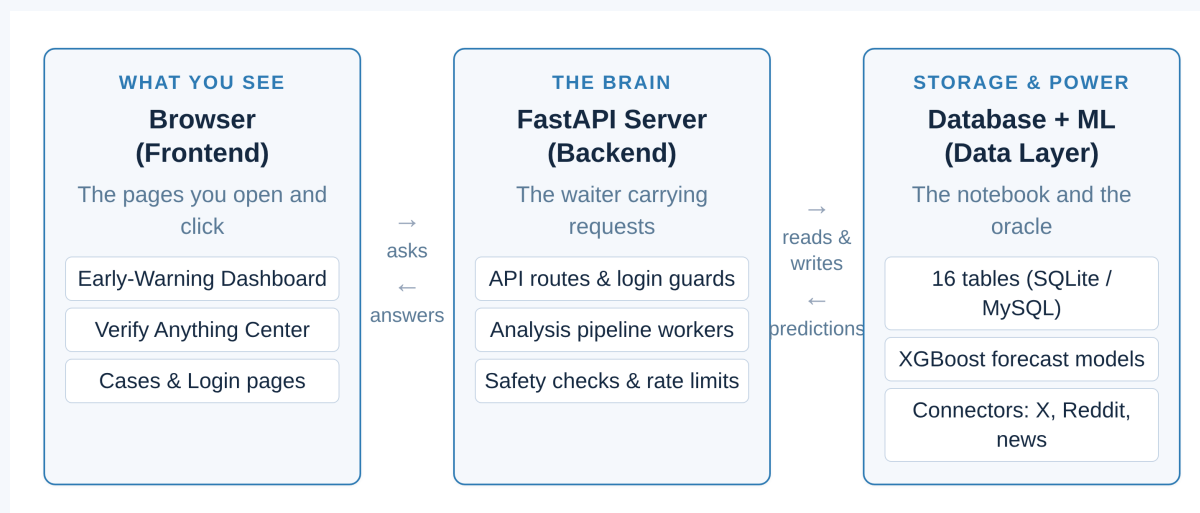


Figure 1: The three tiers of PR•VISION — browser, server, and data layer

Two behind-the-scenes workers make the system feel alive. The **ingestion loop** is a small program that wakes up every few seconds, looks for new posts in the demo feed, and files them into the database, like a clerk collecting letters into a filing cabinet. The **pipeline workers** take each submitted verification job and cook it step by step — extraction, claims, evidence, verdict — reporting progress as they go. Because they work in the background, the website never freezes while a heavy analysis is running; you can keep browsing and come back to the result, exactly like order number 41 being ready while you chat at your table.

In front of everything stands a small traffic officer called a **gateway** (Caddy). It accepts every visit, forwards it to the right server, and adds protective headers on the way out. You never talk to the kitchen directly; the doorman makes sure every request is safe before it goes in.

3. The Toolbox: Technologies We Use

Every craft has its tools, and PR•VISION was built with a deliberately small, well-understood set. The guiding idea was **no magic**: plain HTML, CSS and JavaScript on the front, a modern Python framework at the back, and proven libraries for everything in between. The table below lists the main tools and what each one actually does, in everyday language.

Layer	Tool	Job	In simple words
Frontend	HTML + CSS	Page structure and styling	The walls and paint of the house you see in the browser.
Frontend	Vanilla JavaScript	Interactivity without frameworks	Flips the switches: tabs, forms, live updates — no extra frameworks needed.
Frontend	Chart.js	Drawing graphs	Turns boring numbers into line and bar pictures.
Backend	FastAPI (Python)	The web framework	The waiter: takes requests, serves answers, checks IDs at the door.
Backend	SQLAlchemy	Database toolkit	A translator between Python objects and database rows.
Backend	Uvicorn	Async server	The engine room that keeps the waiter running on many orders at once.
Data	SQLite / MySQL	Storage	The notebook where everything is written down (SQLite for demos, MySQL for production).
Data	Alembic	Migrations	Moves the notebook's layout safely when new columns are added.
ML	XGBoost + joblib	Prediction models	A committee of tiny decision trees that forecasts how far a post will spread.
Docs	ReportLab / pypdf	PDF reports	Prints the final investigation report you can download and share.
Ops	Caddy	Gateway	The doorman: one address in, correctly routed, with safety headers added.
Testing	pytest	Automated tests	A robot that re-checks 123 features after every change so nothing silently breaks.

Table 2: The main tools and their jobs

Why these choices? Because they keep the project **easy to read, easy to run and easy to explain**. A new developer can open the frontend and understand it without learning a build pipeline, and the whole stack runs on any laptop with Python and a browser. When something breaks, there are fewer layers of magic between the symptom and the cause.

4. The Two Main Workspaces

The Early-Warning Dashboard

The dashboard is the **control room**. The moment you log in, it answers four questions with live numbers: how many posts are being monitored, how many are critical, how risky is the average item right now, and how many shares are predicted in the next hour. Below the numbers sit a live alert feed and a trending list, both refreshing automatically. Colour-coded health pills at the top — SYSTEM, DATABASE, PIPELINE, MODEL — tell you in one glance whether the machine itself is healthy, the way a car dashboard tells you the fuel and engine status before you drive.

The VERIFY ANYTHING Center

This is the investigation room. You can drop in **seven kinds of input**: a URL, pasted text, an image, a screenshot, a PDF, a Word document, or a CSV file. The system detects what you gave it, picks the right pipeline, and shows the job's real state — queued, running, with a genuine progress percentage and no fake loading bars. When the analysis is done you get the full report: claims, evidence with citations, media findings, verdicts and scores. A small provider panel at the bottom shows which evidence sources are connected and which are not, so you always know what the conclusion was standing on.

5. The Verification Pipeline Step by Step

When you press **START ANALYSIS**, your content goes on a small journey through four phases. The full map of the trip is drawn as Figure 2 at the end of this chapter; the paragraphs below walk through each phase in plain words. Nothing in this journey is decoration — every box is a real module in the backend code, and the progress you see on screen is the pipeline's actual heartbeat.

Phase 1 • Intake. The system first works out what kind of thing you gave it. A link is fetched politely (respecting robots rules and never stepping behind logins or paywalls), an image is scanned with OCR to read any text inside it, and documents are unpacked page by page — text, tables, links and metadata such as who created the file and when. Tampering signals are noted here too.

Phase 2 • Understand. Long text is useless for checking, so it is cut into **claims** — short sentences that could independently be true or false, like “tap water is contaminated”. The pipeline also tags the people, organisations and topics involved, because knowing a claim is about a city water utility tells the next phase exactly where to hunt for proof.

Phase 3 • Check. Now the honest part. For every claim the system searches legitimate public sources — news agencies, government portals, scientific publishers and professional fact-checking organisations —

and keeps the real URL and date of every hit. Each claim is then compared with its evidence and given one of six verdicts with a confidence level, which Chapter 6 explains fully.

Phase 4 • Predict & Act. Finally the machine-learning models forecast how widely the content will spread in the next 30, 60 and 120 minutes, and everything is rolled into a 0–100 priority score with a written explanation. If the numbers cross a threshold, an alert appears on the dashboard and the analyst can save the whole investigation as a case with one click.

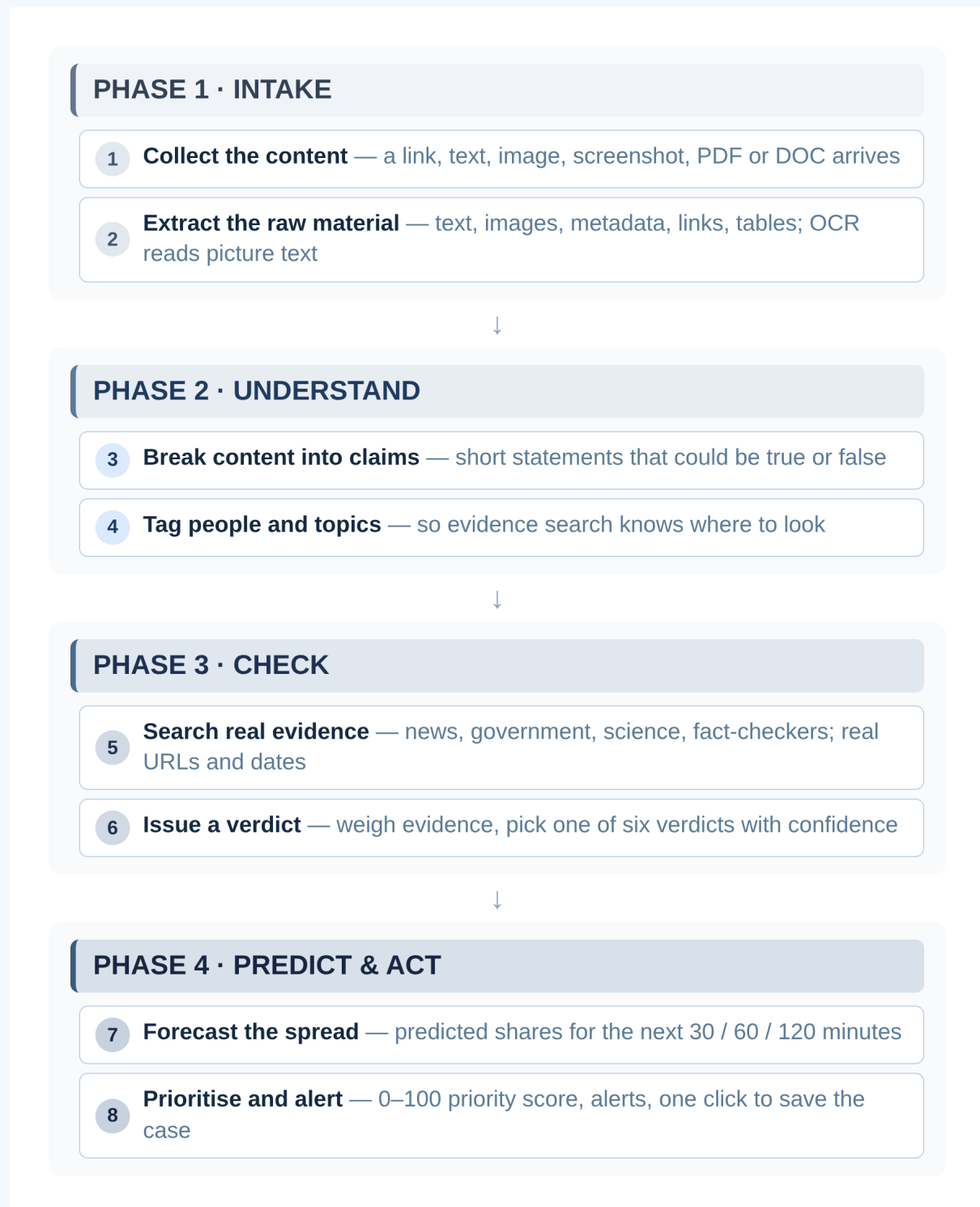


Figure 2: The four-phase verification pipeline, from intake to action

6. The Six Verdicts

Most tools want to shout “FAKE” or “REAL”. PR•VISION refuses, for a simple reason: **absence of evidence is not evidence of absence**. If nobody has published anything about a claim yet, saying “false” would be a guess wearing a scientist's coat. Instead every claim receives one of six careful verdicts, each with a confidence level and the evidence that supports or contradicts it.

Verdict	In plain words	Example
SUPPORTED	Trusted evidence agrees with the claim.	A weather claim matching official records.
UNVERIFIED	No trusted source has spoken on it yet.	A brand-new local rumour, minutes old.
MIXED EVIDENCE	Good sources disagree; the picture is split.	Studies with conflicting findings.
MISLEADING	Built on real facts, but twisted to deceive.	A real photo paired with a false caption.
CONTRADICTED	Trusted evidence directly refutes the claim.	An official statement denying a rumour.
INSUFFICIENT EVIDENCE	There is not enough material to judge fairly.	Only low-quality or unrelated sources found.

Table 3: The six verdicts and what they really mean

Alongside the verdict you always see **why**: the supporting evidence, the contradicting evidence, any professional fact-check matches, and a confidence level from the weighing process. The rule the team lives by is written into the product itself — when the evidence is thin, the platform says so out loud instead of hiding behind a confident guess. An analyst who knows what a machine does not know is far more dangerous to misinformation than one holding a fake 100% score.

7. Predicting Spread with XGBoost

XGBoost is the platform's crystal ball — with an honest label on it. Technically it is a **gradient-boosted tree model**, which in kitchen language means a large committee of very simple decision makers. Each little tree answers a small question (“is the velocity high?”, “is the risk score above 0.7?”), and their votes are combined into one prediction. Thousands of modest opinions add up to one sharp forecast.

The committee learns from history. During training it studies earlier posts — their early share velocity, acceleration, platform, language and risk signals — together with what actually happened afterwards. It

then produces **three forecasts**: expected shares in 30, 60 and 120 minutes. Crucially, training and testing are **split by time**, not randomly: the model is taught on the past and examined on a later period it has never seen, exactly the way a weather forecaster is judged on tomorrow, not on yesterday. This prevents the classic mistake of a model that accidentally peeked at the answers.

Every prediction is shown with **uncertainty** — a range, not a single magic number — and the platform never claims 100% accuracy, because no honest forecaster can. Each trained model is also registered with a version stamp in a model registry, so any number on screen can be traced back to the exact model file that produced it, like a batch number on a medicine bottle.

3 forecast horizons	time-split no peeking at the future	versioned every model traceable
-------------------------------	---	---

8. Alerts, Priority Scores & Cases

Attention is the rarest resource in an intelligence room, so PR•VISION spends it carefully. **Alerts** fire automatically when five kinds of signal appear: a post's misinformation risk climbs, sharing suddenly accelerates, a forecast jumps upward, evidence starts contradicting itself, or media analysis flags a manipulated image. To stop crying wolf, duplicate alerts about the same post within 45 minutes are merged. Every alert carries one of four levels so analysts instantly feel the urgency.

Level	Feeling	Typical trigger
LOW	Worth a glance later	A single mild risk signal on a small post.
MEDIUM	Check it today	Misinformation risk rising steadily on a growing post.
HIGH	Look now	Acceleration spike or forecast jump on a widely shared item.
CRITICAL	Drop everything	High risk plus explosive spread plus conflicting evidence signals.

Table 4: The four alert levels

The **Intervention Priority Score** condenses everything into a single 0–100 number, banded into LOW, MEDIUM, HIGH and CRITICAL. More importantly, the score refuses to be a black box: every number ships with a **WHY THIS SCORE** explanation that lists the ingredients — risk level, spread speed, forecast trend, evidence conflict — and how much each contributed. It is a doctor explaining the diagnosis, not a machine muttering “because computer says 91”.

Finally, an analyst can save any investigation as a **Case**: a folder that snapshots the content, verdicts, evidence, propagation data and scores at that moment, and accepts investigator notes over time. Cases can be monitored, escalated and closed, which turns a fleeting panic into a documented, reviewable investigation.

9. Where the Data Lives

If the platform is a restaurant, the database is its **row of notebooks**: one for staff, one for orders, one for suppliers, one for incidents. PR•VISION keeps sixteen tables, and grouping them into four families makes the whole structure easy to hold in your head. Each row below lists one family, the tables inside it, and what those tables quietly remember.

Family	Tables	What it remembers
People	Users, AuditLogs	Who logged in, what role they hold, and a permanent diary of sensitive actions.
Content	Posts, Claims, Documents, MediaAnalysis	The incoming posts, the claims broken out of them, and what OCR and file analysis found inside images and documents.
Evidence	Sources, FactChecks, Evidence items	Trusted source profiles, professional fact-checks, and every piece of evidence with its URL and date.
Intelligence	PropagationEvents, Predictions, RiskScores, Alerts, Investigations, Cases, ModelVersions	How things spread, what the models forecast, what alerts fired, which cases are open, and which model versions produced the numbers.

Table 5: Sixteen tables, four families

Two quiet details make this notebook trustworthy. First, **indexes** work like the tabs on a filing cabinet, so finding one post among thousands does not require flipping every page. Second, every sensitive action — logins, exports, alert acknowledgements — is written into the audit log, so the question “who did this, and when?” always has an answer.

10. Security in Plain Words

Security sounds scary, but each rule here is really just housekeeping with a good analogy. **Passwords** are never stored as text; they are hashed with PBKDF2 using 390,000 mixing rounds — imagine running the password through a blender 390,000 times. Even the site owner cannot un-blend it back, and stolen blender output is useless on any other site. Login then hands out a **token**, a temporary wristband for the club; the database stores only a hash of each wristband, so even a stolen user table cannot forge one.

Fetching rules protect the platform from being used as a puppet. The URL fetcher refuses internal addresses (the SSRF guard — no asking the website to “fetch” the server's own wiring), requires public internet destinations, respects robots.txt, and never steps behind logins or paywalls. **Rate limits** work like a busy coffee shop door: a fixed number of logins, verifications and exports per minute, so no one can hammer the counter. **Uploads** are checked by magic bytes — the file's actual DNA, not its filename — and size-capped before anything is stored.

Around every page ride **security headers** (CSP, X-Frame-Options, nosniff and friends) that tell browsers to refuse suspicious scripts and framing tricks. Anything fetched from outside — web pages, metadata, OCR text — is treated as **untrusted data** and escaped before display, so a malicious page cannot inject code into the dashboard. And when all else fails, the audit log remembers who did what, which is the security equivalent of leaving the lights on when you leave a room.

11. Real Data vs Demo Mode: Honesty Rules

This chapter is the platform's conscience, and it exists because the fastest way to ruin a verification tool is to let it lie. The project follows three hard rules. First, **every connector tells the truth**: official API integrations exist for X, Reddit, Meta, Instagram, LinkedIn and YouTube, and when credentials are missing the interface plainly shows **API NOT CONFIGURED** instead of quietly pretending. A status panel on every page shows what is live, what is rate-limited and what is disabled.

Second, **demo content always wears a costume**. When the platform runs in demonstration mode, screens and reports are clearly marked **DEMO / SIMULATION**, and the five built-in scenarios — a viral rumour, a doctored screenshot, a suspicious PDF, a fast-spreading story and a conflicting-evidence case — are labelled as the training simulations they are. Nothing generated locally can ever be mistaken for a live capture from the outside world.

Third, **evidence must have a paper trail**. Every citation in a report points to a real, public, legitimately accessed source — a news article, a government page, a scientific paper, a professional fact-check — with its genuine URL and publication date. The system never fabricates a source, never invents a quote, and never scrapes behind authentication walls. If evidence cannot be found, the report says so honestly

and the claim stays UNVERIFIED.

12. Run It Yourself + Project Map

Everything above fits in one folder you can run on your own machine. The steps below take you from an empty terminal to a working platform with demo data, trained models and a login screen. The demo account is **demo@prvision.ai** with password **DemoVision!2026** — it arrives pre-seeded and has administrator rights so you can explore every corner immediately.

- 1. Install Python packages:** create a virtual environment and run *pip install -r backend/requirements.txt*.
- 2. Configure the environment:** copy *.env.example* to *.env*; pick SQLite for a quick demo or MySQL for production, and add any API keys you own.
- 3. Create the database:** run the Alembic migrations so all sixteen tables exist.
- 4. Seed demo data:** run *scripts/seed_demo_data.py* to fill the feed with sample posts and propagation history.
- 5. Train the models:** run *scripts/train_models.py*; it writes the 30/60/120-minute forecast models and registers their versions.
- 6. Start the server:** launch Uvicorn on port 3000, open the site, log in with the demo account, and press START ANALYSIS.

Folder	What lives there
backend/app/api	The route handlers — the doors of the building (auth, posts, verify, cases, alerts, export).
backend/app/models	The sixteen database tables described in Chapter 9.
backend/app/services	The business logic: pipeline orchestration, scoring, alerting — the actual thinking.
backend/app/verification, media, documents	Claim checking, image/OCR analysis, PDF and DOCX extraction.
backend/app/propagation, forecasting	Spread metrics and the XGBoost prediction layer.
backend/app/connectors	The six official platform integrations with honest status reporting.
frontend	Plain HTML/CSS/JS pages: landing, dashboard, verify, cases, login, report.
ml	Training scripts, datasets and the versioned model files (.joblib).
scripts	Helpers: seeding, training, migrations, the self-healing start daemon.
docs	This guide's screenshots and additional documentation.

Table 6: The project folder map

A final word on limits, because a tool that hides its limits eventually betrays you. PR•VISION sees only what public sources and official APIs allow; it cannot read private groups or encrypted messages. Its forecasts are probabilities, not promises. Its verdicts reflect the evidence that exists today and may change tomorrow. Within those honest borders, it does exactly what it claims: notice early, check carefully, predict cautiously, and point the analyst at the fire that matters most.